

A Survey of Language Models for Source-level Code Optimisation

Sam Armstrong

Abstract—With software struggling to keep up with rapid hardware advances, code optimisation has become increasingly critical. While traditional compilers excel at low-level optimisations like instruction scheduling, they often miss higher-level semantic optimisations that require contextual understanding. This has led to the emergence of source-level code optimisation using language models (LMs). In this critical analysis, we survey this rapidly evolving literature, highlighting a convergence towards approaches that integrate LMs with developer-style tooling, such as static and dynamic analysis. However, significant gaps remain in reliability, language coverage, and real-world applicability, while the impacts of optimisations on maintainability are underexplored. We suggest future directions, including multi-language benchmarks, frameworks that combine LM optimisation with human review, and unified systems that integrate techniques such as reinforcement learning to produce performant yet maintainable code.

I. INTRODUCTION

One of the primary reasons for pursuing better optimisation across the software stack is the ‘software gap’ [1] where software is increasingly unable to fully take advantage of the compounding year over year performance improvements in hardware.

The traditional process of optimising code is twofold; the programmer manually optimising the source code, then the compiler optimising it while maintaining the same functionality [2]. Compilers are strong at low-level optimisations, such as instruction scheduling and loop transformations [3], but they achieve their optimisations through heuristics that can be brittle to rapidly evolving hardware architectures [1] and miss higher-level optimisations that are often the most critical for program performance. These include things like algorithmic restructuring and refactoring, which require more complex contextual understanding than could be handled by heuristic techniques.

The limitations of these two levels of optimisation have led to the emergence of automated source-level optimisation as a third way to improve software efficiency [2], which works by optimising the high-level source code before it’s compiled.

The source-level optimisation paradigm is a particularly promising domain for language models (LMs), as they can learn where and how different optimisation patterns apply, then apply this in new codebases, being able to take into account the context that compiler heuristics would miss [3]. This has been demonstrated in practice, with recent systems showing speedups of up to $21.5\times$ in real-world cases [4], proving that

source-level optimisation has the potential to exceed traditional compiler performance.

However, applying LMs to code optimisation raises significant challenges. Unlike traditional compilers, which guarantee semantic preservation (keeping the meaning of the code identical), LMs frequently generate incorrect code and require verification to be useful in practice [3].

We review the state of the art in source-level code optimisation, identifying the key findings, challenges, and opportunities for future research in the field.

II. RELATED WORK

A. An Overview of Language Models for Code Optimisation

Gong et al., from early 2025, survey how both general-purpose and code-specialised language models are applied to code optimisation [3]. They explain the key benefits of using language models for code optimisation, including ‘their deep semantic understanding of code’ and the capacity of these models to explore the optimisation space, both of which help complex optimisations to be identified and implemented. They also identify several limitations of language models in this context, including the high cost of training and inference, and the difficulty of ensuring the correctness and optimality of generated code changes.

Another key insight from this survey was the identification of key challenges that reflect open problems in the field, such as:

- Balancing model size and intelligence with cost and inference speed
- Enabling models to generalise optimisation capabilities across programming languages
- Improving the currently limited trust and reliability of generated code

As part of the survey, they derive several potential future research directions from these challenges, including:

- Experimenting with model compression or small LM ensembles to balance cost and intelligence
- Establishing standardised real-world benchmarks to evaluate model performance
- Using RLHF to better leverage human expert techniques

Another study, Rosas et al. [5], compares classical (non-LM) source-level optimising compilers against state of the art Large Language Models (LLMs) on high-level parallelisation

tasks, finding that CodeLlama-70B achieved speedups of up to $1.75\times$, whereas the incumbent CETUS achieved $1.67\times$. To accomplish this, CodeLlama-70B was combined with automated correctness and performance evaluation frameworks.

Yi et al. [6] introduce PerfOpt, an evaluation set of 65 real-world performance optimisation tasks from major open-source Java projects, as part of their study on using LLMs for optimisation - which yields useful benchmark results that we'll mention later.

B. NMT for Code Optimisation

One of the earliest highly successful applications of language models to code optimisation was SUPERSONIC, introduced by Chen et al. [2]. This uses a sequence-to-sequence NMT (encoder-decoder) model that generates code changes for C/C++ source code based on code diffs, rather than rewriting entire functions from scratch. This approach addressed the issue of LMs struggling with larger code contexts, as it limited the size of the code the model was attempting to optimise. Despite being drastically smaller, SUPERSONIC improved performance over the general-purpose LLMs available at the time (GPT-3.5-Turbo and GPT-4).

C. Prompt-Guiding LLMs

Since the introduction of SUPERSONIC [2], decoder-only LLMs have become the standard for code optimisation, due to their strong reasoning capabilities and the wide availability of high quality pretrained models. There have been several systems and techniques proposed for guiding these non-specialised models with contextual prompts to improve the performance and correctness of code optimisations they generate.

1) *Execution Feedback*: One example of such a system is PERFCODEGEN [7]. The key insight with PERFCODEGEN is the use of execution feedback, which prompts the LLM with the results of unit tests to allow it to iteratively refine its optimisations. This process runs in two stages, the first where the unit tests are only concerned about correctness, the second about performance so bottlenecks can be removed while preserving behaviour. Experiments on Python benchmarks showed that this simple feedback loop allowed much smaller models (Phi-3-mini) to surpass the performance of larger models (GPT-4).

2) *Static and Dynamic Analysis*: Kim et al. [8] introduce ECO, which constructs 'performance-aware' prompts using static and dynamic analysis around the input code. It achieves this by running a symbolic advisor to propose potential optimisation hints, then retrieving runtime optimisation instructions from a database of past examples whose performance characteristics match the current code, combining these into a final prompt. This process steers the LLM towards faster implementations, achieving speedups of up to $7.81\times$.

Static analysis is also employed in the SemOpt framework [9], which uses machine learning to summarise and cluster recurring optimisation strategies, derives pattern-matching

rules to detect where each strategy applies, then prompts an LLM with these strategies to rewrite the code at the corresponding locations. On an evaluation on several large open-source systems (including Redis and LevelDB), SemOpt achieves average performance improvements between 1.68% and 10.4%, with the maximum performance improvement being 218.07%.

3) *Search-based Optimisation*: Gao et al. [10] propose a different method of guiding an LLM - treating code optimisation as a search problem. Starting from the existing solution, their framework, SBLLM, iteratively prompts an LLM to propose variants, executes them to measure correctness and speed, and selects samples. Inspired by evolutionary algorithms, the LLM is then prompted to 'mutate' strategies across these samples, to produce the next generation. This system achieves substantial speedups on Python and C++ benchmarks, even in some cases producing faster code than reference implementations.

D. Fine-tuning with Reinforcement Learning

Reinforcement learning (RL) recently emerged as a strong technique for improving the performance of code LLMs [11]. Duan et al. [12] followed this trend to train a small autoregressive language model with RL to optimise Python code. Their framework, PerfRL, uses a reward mechanism based on compilation success, runtime error checks, and execution time improvements to fine-tune the model. On the PIE dataset [13], PerfRL matches or outperforms much larger models while being trained on only a single GPU. This highlights the efficacy of smaller models for high-level code optimisation when they're specially tuned for the task. It's worth noting, however, that the model was only trained and tested on Python code, so it remains unclear how well this approach could generalise to other languages.

E. Multi-Model Orchestration

Drawing on their study of 17 LLMs, Chen et al. [14] observe that no single model dominates across all programming languages or task categories; a model that is excellent at generating optimisations in one language may be weaker at optimising code in another. In response, they created PerfOrch, a system which decomposes each optimisation task into a generation, bug fixing, and performance refinement stage. It then routes each stage to the model shown to work best for that language and problem type. On benchmarks such as EFFIBENCH-X [15], this multi-model orchestration achieves better performance than any individual LLM.

F. Larger Scale Optimisation

Most of the papers discussed up to this point solely focus on the optimisation of isolated functions or code snippets, where the model is only optimising a relatively small, self-contained piece of code. In realistic software projects, however, performance problems are often emergent properties of complex call graphs, so optimising isolated functions is not enough.

POLO [4] is a project-level optimisation framework explicitly designed for this setting. It starts by profiling a full system and building a ‘Program Call Graph’ that captures how functions interact at runtime. POLO then constructs a ‘Program Structure Graph’ to give the LLM a deeper understanding of the class and function hierarchy. With this context, an LLM proposes source-level code changes, while a second LLM tests and benchmarks the resulting program. Experiments on open-source systems such as MiniSQL and SkipList show speedups ranging from $1.34\times$ to $21.5\times$.

III. ANALYSIS

The papers surveyed demonstrate both the strengths and limitations of language models for optimising source-level code, and reflect a rapidly evolving field, with the majority of papers cited being published in 2024 or 2025.

A. Strengths and Progress of the Field

There has been a convergence of approach in recent papers, with ECO, SemOpt and POLO [4], [8], [9] all using static or dynamic analysis to guide LLMs through prompt engineering. This approach of integrating techniques traditionally used manually by developers into LLM pipelines, reflects a clear evolution from single-shot prompting towards structured systems with feedback loops, program graphs, and search over candidate solutions. This convergence suggests the field is beginning to move towards a consensus on effective methodologies.

The field has also made meaningful progress towards practicality and scalability. As noted in Section II, most early systems focused on isolated functions, but POLO now directly tackles project-level optimisation. POLO’s use of program call graphs and program structure graphs [4] specifically addresses the fact that performance problems in real software often emerge from complex call graphs rather than individual functions. Evaluations on production systems such as Redis, RocksDB, and full Java projects [6], [9] also give more credibility to the reported results than benchmarks alone.

Several papers directly address the cost/intelligence trade-off identified by Gong et al. [3] as a key challenge for the field. SUPERSONIC [2] demonstrated early on that a small, domain-specific encoder-decoder model could outperform GPT-3.5 and GPT-4 on code optimisation tasks. Likewise, PerfRL’s reinforcement learning approach [12] matches larger models with a much smaller model trained on a single GPU, and PERFCODEGEN [7] shows that execution feedback enables Phi-3-mini to surpass GPT-4. These results suggest that targeted training and advanced tooling can compensate for reduced model size, making source-level code optimisation more accessible and cost-effective.

B. Weaknesses and Gaps in the Literature

Despite these advances, the surveyed papers share several key limitations that have not been addressed. For example, systems such as PERFCODEGEN, SBLLM, PerfRL, and POLO [4],

TABLE I
PROGRAMMING LANGUAGES AND EVALUATION SCALE

System	Languages	Scale
SUPERSONIC [2]	C, C++	Function
PERFCODEGEN [7]	Python	Function
ECO [8]	C++	Function
SemOpt [9]	C, C++	Project
SBLLM [10]	Python, C++	Function
PerfRL [12]	Python	Function
PerfOrch [14]	Python, Java, C++, Go, Rust	Function
POLO [4]	C, C++	Project
PerfOpt [6]	Java	Project

[7], [10], [12] use a research methodology assuming that improving benchmark runtimes implies the same speedups will be seen in real-world performance. As PerfOpt’s [6] results suggest, this assumption does not always hold; code changes that improve benchmark results may still underperform or miss deeper optimisations.

Another limitation is the lack of generalisation and testing across multiple programming languages. The majority of systems discussed only target Python or C/C++, as can be seen in Table I. This was one of the key challenges raised by Gong et al. [3], and suggests that the generalisation of language models for code optimisation across languages remains largely unexplored.

One area that is notably underexplored is the reliability and regression risk of LMs. As Rosas et al. [5] identify, LLMs struggle significantly with larger contexts and often produce incorrect code. Systems such as POLO [4] and PERFCODEGEN [7] mitigate this through test-based verification loops, but a risk remains that subtle bugs get introduced that are not caught by the test suite. Furthermore, as shown in Table II, only 62% of LLM-generated patches compile and pass tests [6], indicating that reliability remains a barrier to real-world adoption.

Similarly, source-level optimisations by LMs have the potential to reduce the readability and maintainability of code. Many of the systems discussed that aim to optimise performance above all else, such as SBLLM [10], may produce code that is correct and fast but difficult for developers to understand or modify. Systems that could rewrite entire parts of the codebase, such as POLO [4], could be particularly problematic, yet this is not well-explored in the literature.

Another gap in the literature is that optimisation collaboration, where source-code optimisers work in tandem with developers, lacks systematic exploration compared to fully autonomous optimisation systems. PerfOpt’s finding that explicit human-written problem descriptions dramatically improve results [6] suggests there is substantial room for LLMs as optimisation suggesters that can be reviewed and refined by developers.

C. Contradictions

One notable contradiction in the literature is how to balance the trade-off between model size and capabilities. PerfRL and SUPERSONIC argue that compact models that utilise

TABLE II
HUMAN VS. LLM PERFORMANCE ON REAL-WORLD JAVA OPTIMIZATION
TASKS FROM PERFOPT (% OF SUCCESSFUL CASES) [6]

Metric	Human	Best LLM (Gemini)
Significant Improvement ($>1.5\times$)	62.79%	46.23%
Valid Code Changes	100%	62%

specialised training can rival or beat very large LLMs, supporting the goal of reducing cost. At the same time, PerfOrch [14] takes the exact opposite approach, embracing the power of coordinating multiple powerful models to improve the correctness and performance of optimisations at the expense of increased compute, memory, and cost requirements.

Another contradiction exists between the high speedups reported in case studies and the more modest results observed in broad, contamination-resistant benchmarks. Papers like POLO, ECO, and SBLLM [8]–[10] report substantial speedups on their chosen tasks, implying that LLMs can outperform strong compiler baselines. However, EFFIBENCH-X [15] and PerfOpt [6] show that, on broader or more realistic benchmarks, LLMs still lag behind human experts and rarely discover optimal solutions. This suggests that the specialised benchmarks may not be representative of real-world performance for the systems that use them.

While 62% of LLM-generated optimisations compiled and passed the tests, human optimisations achieved $>1.5\times$ speedup in 62.79% of cases compared to only 46.23% for the best-performing LLM. This suggests that human experts remain substantially better at discovering optimal solutions for complex real-world performance problems.

IV. FUTURE WORK

Based on the analysis, there are several promising directions for future work.

A. Language Coverage and Evaluation

As highlighted in Table I, the majority of systems discussed focus on Python or C/C++. This was identified as one of the key challenges by Gong et al. [3], and indicates that the generalisation of code optimisation techniques across languages remains underexplored. This is particularly true for smaller models like PerfRL [12], which demonstrated strong performance on Python benchmarks, but with no evidence for or against this same technique working for other languages, or multiple languages. A key question is whether techniques that work well for Python (like PerfRL) transfer to statically-typed languages like Rust or Java. Future work could evaluate existing systems across languages to understand where they transfer successfully and where they fail.

In a similar vein, there is a lack of standardised, real-world benchmarks for code optimisation across multiple languages. As discussed in Section III, many systems assume that improving benchmark runtimes implies real-world speedups, an assumption that PerfOpt’s results suggest is not always true

[6]. Developing multi-language benchmarks that cover diverse language paradigms and runtime characteristics would help address both of these gaps, enabling more rigorous evaluation and encouraging research into cross-language optimisation techniques.

B. Automated vs Human-supervised Optimisation

There is a gap in the literature around developers using LLMs as optimisation assistants. Systems such as SBLLM, POLO, and PerfOrch push towards fully automated pipelines, yet PerfOpt’s results show that human experts still substantially outperform LLMs at discovering optimal solutions, as shown in Table II.

Future work could explore using LM source-level optimisers as developer tools rather than automated agents. For example, small models could run locally during development to suggest optimisations for immediate review. This approach could help address the challenges of robustness, maintainability, and reliability identified in the literature, as human oversight could catch regressions or problematic changes before they are committed.

C. Unified Optimisation Frameworks

The analysis identified some convergence in recent papers, with systems increasingly combining prompt engineering techniques, like static analysis and feedback loops, with pretrained LLMs. However, there were several other methodologies applied with success, like reinforcement learning (as in PerfRL [12]), and model ensembles (as in PerfOrch [14]). Future work could explore unified frameworks that combine these methodologies to evaluate their combined performance.

Such a unified framework could leverage the strengths of each approach - for example, using static analysis to identify optimisation opportunities, retrieval to provide relevant examples, program graphs to capture project-level context, and an ensemble of models fine-tuned with reinforcement learning according to their task in the pipeline. Studies comparing different combinations of these techniques could help identify which are complementary.

D. Reliability and Maintainability

Our analysis pointed out that reliability remains a significant barrier to real-world adoption of LM-based code optimisation. Future work could explore more robust verification techniques beyond unit testing, such as formal verification [16], to provide stronger correctness guarantees for generated optimisations, which could help broader adoption of LM-based source-level code optimisation.

Similarly, maintainability of optimised code has received little attention in the literature. Future research could develop metrics for measuring the maintainability impact of optimisations, and explore techniques that balance performance gains against code readability, such as constraining optimisations to preserve coding style or training models with RLHF to prefer more

readable code. Moreover, a survey of the impact of optimisations proposed by the existing systems on maintainability would be beneficial to understand this trade-off and how it should be addressed.

V. CONCLUSION

In this critical analysis, we surveyed language models for source-level code optimisation, identifying convergence toward approaches that integrate LMs with developer tooling. The literature demonstrates that LMs can achieve substantial speedups with the help of additional context, such as static and dynamic analysis, or using other methods like reinforcement learning. However, it also reveals that significant gaps remain in reliability, language coverage, and real-world applicability, which are open problems that we suggest be explored in future work.

REFERENCES

- [1] Z. Wang and M. O’Boyle, “Machine learning in compiler optimisation,” *Proceedings of the IEEE*, vol. 106, no. 11, pp. 1879–1901, 2018.
- [2] Z. Chen, S. Fang, and M. Monperrus, “Supersonic: Learning to generate source code optimizations in c/c++,” *IEEE Transactions on Software Engineering*, vol. 50, no. 11, p. 2849–2864, Nov. 2024. [Online]. Available: <http://dx.doi.org/10.1109/TSE.2024.3423769>
- [3] J. Gong, V. Voskanyan, P. Brookes, F. Wu, W. Jie, J. Xu, R. Giavrimis, M. Basios, L. Kanthan, and Z. Wang, “Language models for code optimization: Survey, challenges and future directions,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.01277>
- [4] J. Bai, R. Xu, S. Wu, D. Yang, J. Zhao, and G. Chen, “Polo: An llm-powered project-level code performance optimization framework,” in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, J. Kwok, Ed. International Joint Conferences on Artificial Intelligence Organization, 8 2025, pp. 7319–7328, main Track. [Online]. Available: <https://doi.org/10.24963/ijcai.2025/814>
- [5] M. R. Rosas, M. T. Sanchez, and R. Eigenmann, “Should ai optimize your code? a comparative study of classical optimizing compilers versus current large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.12146>
- [6] L. Yi, G. Gay, and P. Leitner, “An experimental study of real-life llm-proposed performance improvements,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.15494>
- [7] Y. Peng, A. D. Gotmare, M. Lyu, C. Xiong, S. Savarese, and D. Sahoo, “Perfcodegen: Improving performance of llm generated code with execution feedback,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.03578>
- [8] S.-H. Kim, J. Hahn, S. Cha, and Y.-S. Han, “Eco: Enhanced code optimization via performance-aware prompting for code-llms,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.10517>
- [9] Y. Zhao, Y.-A. Xiao, Q. Xiao, Z. Zhang, and Y. Xiong, “Semopt: Llm-driven code optimization via rule-based analysis,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.16384>
- [10] S. Gao, C. Gao, W. Gu, and M. Lyu, “Search-based llms for code optimization,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.12159>
- [11] J. Wang, Z. Zhang, Y. He, Z. Zhang, X. Song, Y. Song, T. Shi, Y. Li, H. Xu, K. Wu, X. Yi, Z. Wan, X. Yuan, Z. Wang, K. Lu, M. Huo, T. Jingqun, G. Qian, K. Li, Q. Chen, and L. He, “Enhancing code llms with reinforcement learning in code generation: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.20367>
- [12] S. Duan, N. Kanakaris, X. Xiao, H. Ping, C. Zhou, N. K. Ahmed, G. Ma, M. Capota, T. L. Willke, S. Nazarian, and P. Bogdan, “Perfrl: A small language model framework for efficient code optimization,” 2025. [Online]. Available: <https://arxiv.org/abs/2312.05657>
- [13] A. Shypula, A. Madaan, Y. Zeng, U. Alon, J. Gardner, M. Hashemi, G. Neubig, P. Ranganathan, O. Bastani, and A. Yazdanbakhsh, “Learning performance-improving code edits,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=ix7rL0oHok>
- [14] H. Chen, Z. Qi, H. Li, H. Chen, J. Chen, K. Peng, I. K. Kim, K. H. Lee, and S. He, “Beyond single llms: Enhanced code generation via multi-stage performance-guided llm orchestration,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.01379>
- [15] Y. Qing, B. Zhu, M. Du, Z. Guo, T. Y. Zhuo, Q. Zhang, J. M. Zhang, H. Cui, S.-M. Yiu, D. Huang, S.-K. Ng, and L. A. Tuan, “Effibench-x: A multi-language benchmark for measuring efficiency of llm-generated code,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.13004>
- [16] S. Bhatia, J. Qiu, N. Hasabnis, S. A. Seshia, and A. Cheung, “Verified code transpilation with llms,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.03003>